

Date: 13/07/2026

Experiment - 04(A)

Minimax Algorithm

Aim: To implement the Minimax algorithm to determine the optimal value from a set of leaf node values based on a user-defined tree depth and root node type.

Algorithm:

1. Start the program.
2. Read the depth of the game tree from the user.
3. Calculate the number of leaf nodes as 2^{depth} .
4. Read whether the root node is a MAX or MIN node.
5. Read the required leaf node values from the user.
6. Start the Minimax algorithm from depth 0 using the selected root node type.
7. If the current depth reaches the given depth, return the corresponding leaf node value.
8. If the current node is a MAX node, recursively evaluate its child nodes and return the maximum value.
9. If the current node is a MIN node, recursively evaluate its child nodes and return the minimum value.
10. Display the evaluation flow of each node.
11. Alternate between MAX and MIN nodes at each level of the tree.
12. Continue the process until the root node is evaluated.
13. Print the optimal value obtained from the Minimax algorithm.
14. End the program.

Code

```
1 def minimax(depth, index, isMax):
2     if depth == max_depth:
3         return values[index]
4
5     left = minimax(depth + 1, index * 2, not isMax)
6     right = minimax(depth + 1, index * 2 + 1, not isMax)
7
8     if isMax:
9         result = max(left, right)
10        print("    " * depth +
11              f"MAX({left}, {right}) = {result}")
```

```
12     else:
13         result = min(left, right)
14         print(" " * depth +
15               f"MIN({left}, {right}) = {result}")
16
17     return result
18
19
20 max_depth = int(input("Enter the depth of the tree: "))
21
22 node_type = input(
23     "Enter root node type (MAX/MIN): "
24 ).upper()
25
26 while node_type not in ["MAX", "MIN"]:
27     print("Please enter either MAX or MIN.")
28     node_type = input(
29         "Enter root node type (MAX/MIN): "
30     ).upper()
31
32 leaf_count = 2 ** max_depth
33 values = []
34
35 print(f"Enter {leaf_count} leaf node values:")
36
37 for i in range(leaf_count):
38     values.append(
39         int(input(f"Value {i + 1}: "))
40     )
41
42 isMax = (node_type == "MAX")
43
44 print("\nMinimax Evaluation Flow:")
45 print("-----")
46
47 result = minimax(0, 0, isMax)
48
49 print("-----")
50 print("Optimal Value =", result)
```

Sample Output:

```
1 Enter the depth of the tree: 3
2
3 Enter root node type (MAX/MIN): MIN
4
5 Enter 8 leaf node values:
```

```
6
7 Value 1: 3
8 Value 2: 2
9 Value 3: 8
10 Value 4: 6
11 Value 5: 4
12 Value 6: 9
13 Value 7: 5
14 Value 8: 7
15
16 Minimax Evaluation Flow:
17 -----
18
19     MAX(3, 2) = 3
20     MAX(8, 6) = 8
21     MIN(3, 8) = 3
22
23     MAX(4, 9) = 9
24     MAX(5, 7) = 7
25     MIN(9, 7) = 7
26
27     MIN(3, 7) = 3
28
29 -----
30 Optimal Value = 3
```

Result: The Minimax algorithm was successfully implemented with a user-defined tree depth and root node type. The evaluation flow was displayed and the optimal value was obtained.

Date: 20/07/2026

Experiment - 04(B)

Alpha-Beta Pruning

Aim: To implement the Alpha-Beta Pruning algorithm to determine the optimal value while reducing unnecessary node evaluations based on a user-defined tree depth and root node type.

Algorithm:

1. Start the program.
2. Read the depth of the game tree from the user.
3. Calculate the number of leaf nodes as 2^{depth} .
4. Read whether the root node is a MAX or MIN node.
5. Read the required leaf node values from the user.
6. Initialize alpha to negative infinity and beta to positive infinity.
7. Start the Alpha-Beta Pruning algorithm from depth 0 using the selected root node type.
8. If the current depth reaches the given depth, return the corresponding leaf node value.
9. If the current node is a MAX node, recursively evaluate its child nodes and update alpha with the maximum value.
10. If alpha becomes greater than or equal to beta, prune the remaining branches.
11. If the current node is a MIN node, recursively evaluate its child nodes and update beta with the minimum value.
12. If beta becomes less than or equal to alpha, prune the remaining branches.
13. Display the evaluation flow of each node.
14. Continue the process until the root node is evaluated.
15. Print the optimal value obtained using Alpha-Beta Pruning.
16. End the program.

Code

```
1 def alpha_beta(depth, index, isMax, alpha, beta):  
2  
3     if depth == max_depth:  
4         return values[index]  
5
```

```
6     left = alpha_beta(  
7         depth + 1,  
8         index * 2,  
9         not isMax,  
10        alpha,  
11        beta  
12    )  
13  
14    if isMax:  
15        best = left  
16        alpha = max(alpha, best)  
17  
18        right = alpha_beta(  
19            depth + 1,  
20            index * 2 + 1,  
21            not isMax,  
22            alpha,  
23            beta  
24        )  
25  
26        best = max(best, right)  
27  
28        if beta <= alpha:  
29            print(" " * (depth + 1) +  
30                "Right branch pruned")  
31  
32        print(" " * depth +  
33            f"MAX({left}, {right}) = {best}")  
34  
35        return best  
36  
37    else:  
38        best = left  
39        beta = min(beta, best)  
40  
41        right = alpha_beta(  
42            depth + 1,  
43            index * 2 + 1,  
44            not isMax,  
45            alpha,  
46            beta  
47        )  
48  
49        best = min(best, right)  
50  
51        if beta <= alpha:  
52            print(" " * (depth + 1) +
```

```
53         "Right branch pruned")
54
55     print(" " * depth +
56           f"MIN({left}, {right}) = {best}")
57
58     return best
59
60
61 max_depth = int(input("Enter the depth of the tree: "))
62
63 node_type = input(
64     "Enter root node type (MAX/MIN): "
65 ).upper()
66
67 while node_type not in ["MAX", "MIN"]:
68     print("Please enter either MAX or MIN.")
69     node_type = input(
70         "Enter root node type (MAX/MIN): "
71     ).upper()
72
73 leaf_count = 2 ** max_depth
74 values = []
75
76 print(f"Enter {leaf_count} leaf node values:")
77
78 for i in range(leaf_count):
79     values.append(
80         int(input(f"Value {i + 1}: "))
81     )
82
83 isMax = (node_type == "MAX")
84
85 print("\nAlpha-Beta Evaluation Flow:")
86 print("-----")
87
88 result = alpha_beta(
89     0,
90     0,
91     isMax,
92     float('-inf'),
93     float('inf')
94 )
95
96 print("-----")
97 print("Optimal Value =", result)
```

Sample Output:

```
1 Enter the depth of the tree: 3
2
3 Enter root node type (MAX/MIN): MIN
4
5 Enter 8 leaf node values:
6
7 Value 1: 7
8 Value 2: 4
9 Value 3: 8
10 Value 4: 6
11 Value 5: 3
12 Value 6: 9
13 Value 7: 5
14 Value 8: 2
15
16 Alpha-Beta Evaluation Flow:
17 -----
18
19     MIN(7, 4) = 4
20     MIN(8, 6) = 6
21     MAX(4, 6) = 6
22
23     MIN(3, 9) = 3
24     MIN(5, 2) = 2
25     MAX(3, 2) = 3
26
27 MIN(6, 3) = 3
28
29 -----
30 Optimal Value = 3
```

Result: The Alpha-Beta Pruning algorithm was successfully implemented with a user-defined tree depth and root node type. The evaluation flow was displayed and the optimal value was obtained while eliminating unnecessary node evaluations when possible.